# Lesson 2: Key Press Reactor

Unity

VIDEO GAME CREATION

This week: sequences, events, and the game loop. Your job is to build something that reacts when you press a key - and this is also the start of your Semester 1 project, so save it somewhere safe and give it a name you'll recognize.

*Tip: this lesson's Assets folder already has a ready-to-use placeholder character image (and an optional sound effect for Level 3) - you don't need to find or make your own art today.*

## Setup

1. Create a new 2D project named [YourName]Game.
2. In the Hierarchy, right-click > Create Empty, name it Player, add a Sprite Renderer component and drag one of the placeholder images from this lesson's Assets folder (e.g. player_circle_blue.png) into its Sprite field.

## Level 1 - Minimal (MVP)

*One key press causes one visible reaction. This is a complete finish for today - reaching this is enough to move on to next week.*

3. With Player selected, Add Component > New Script, name it PlayerMove, Create and Add.
4. Double-click the script to open it. Add using UnityEngine.InputSystem; as a new line right after the existing using UnityEngine; at the top - this is what makes Keyboard.current available.
5. Replace the Update() method body with the code below.
6. Save, return to Unity, press Play, and use the arrow keys.

```
void Update() {
    float h = 0f;
    if (Keyboard.current.rightArrowKey.isPressed) h = 1f;
    if (Keyboard.current.leftArrowKey.isPressed) h = -1f;
    transform.Translate(new Vector3(h, 0, 0) * 3f * Time.deltaTime);
}
```

## Alternate Path: Visual Scripting (node graph, no typing)

*Unity's built-in Visual Scripting package lets you build the same movement logic as a node graph instead of C# code, using the same new Input System as the code path above - not the legacy Input Manager.*

7. With Player selected, Add Component > Input > Player Input (Unity installs the Input System package on first use if prompted). In its Actions field, click Create Actions and save the new asset as PlayerControls.
8. Double-click PlayerControls to open the Input Actions editor. Under the Player action map, select the existing Move action and edit its 2D Vector Composite bindings so Up/Down/Left/Right point at the Up Arrow, Down Arrow, Left Arrow, and Right Arrow keys (matching every other engine in this lesson), then click Save Asset.
9. In Edit > Project Settings > Visual Scripting, click Regenerate Nodes (only needed once per project) so the Input System event nodes become available in the fuzzy finder.
10. In Edit > Project Settings > Input System Package, click Create Settings Asset if you haven't already (also only needed once per project).
11. With Player selected, Add Component > Visual Scripting > Script Machine (Unity installs the Visual Scripting package on first use if prompted), click the graph field to create a new Script Graph asset, name it PlayerMoveVisual, and click Edit Graph to open it.
12. Right-click the canvas, search for and add an "On Input System Event Vector 2" node. Leave Target as This, set Input Action to Move, and set Input Action Change Type to On Hold.
13. Add a "Vector 2 > Get X" node and wire the Vector 2 Value output into it.
14. Add a "Vector 3 > Construct Vector 3" node and wire Get X's output into its X input.
15. Add a "Transform > Translate" node; wire the On Input System Event Vector 2 node's Trigger output into it, and wire the constructed Vector 3 into its Translation input.
16. Save the graph (Ctrl+S), press Play, and test with the arrow keys - same result as the code path.

*Level 2: add a "Vector 2 > Get Y" node wired to the same On Input System Event Vector 2 node's Vector 2 Value output, and wire it into the Vector 3's Y input alongside Get X into X - the Move action already carries both axes, you're just choosing to use more of its output. Level 3's five goals (blink, beep-beep sound, recolor, clamp, and grouping it all under one Branch node keyed off whether you're moving) are all reachable with more nodes from the Sprite Renderer, Audio, and Math categories - same ideas as the code path, just wired instead of typed.*

## Level 2 - Complete (Alpha/Beta)

*All 4 arrow keys move your character in 4 directions.*

- Add vertical movement too: same if-pattern with Keyboard.current.upArrowKey and Keyboard.current.downArrowKey building a float v, then include it in the Vector3 as (h, v, 0).

## Level 3 - Refined (Ready to Publish) - pick any that interest you

- Animate: make the sprite blink while moving, and stay solid (visible) when idle - toggle spriteRenderer.enabled based on a timer.
- Sound: play sfx_blip.wav in a beep-beep pattern while moving, silent when idle - add an AudioSource, check audioSrc.isPlaying before calling Play() again, and call Stop() when idle.
- Recolor: swap in a different placeholder image for each direction (spriteRenderer.sprite), rather than tinting one - assign a different Sprite per direction in the Inspector, and back to your default/orange sprite when idle.
- Clamp: keep the Player's position on screen using Mathf.Clamp().

- Structure: keep all four of the above together in the same Update() method - not scattered across separate functions.

## If Something's Wrong

- Script errors show in red in the Console - fix the top one first; others often resolve on their own.
- If nothing moves, confirm the Player GameObject wasn't deleted/duplicated and the script component is enabled (checkbox visible in the Inspector).
- "The type or namespace name 'Keyboard' could not be found": confirm using UnityEngine.InputSystem; was added at the top of the script, not just using UnityEngine;.
- NullReferenceException on Keyboard.current, or nothing responds to key presses at all: check Edit > Project Settings > Player > Other Settings > Active Input Handling - a fresh Unity project defaults to "Both," which supports this code, but if it's been changed to "Input Manager (Old)" only, switch it to "Both" or "Input System Package (New)" and let Unity restart the Editor when prompted.
- Visual Scripting: no Input System nodes show up when you search the fuzzy finder - Edit > Project Settings > Visual Scripting > Regenerate Nodes hasn't been run yet (one-time setup, not something you repeat every lesson).
- Visual Scripting: Move doesn't appear in the On Input System Event Vector 2 node's Input Action dropdown - the Player Input component's Actions field is still empty; assign it the PlayerControls asset.

## Want More?

learn.unity.com, Student Resource List > Unity